从特征供给瓶颈，到自适应在线训练

核心问题：特征不能全量驻留时，怎样让 GPU 持续获得训练数据？

Vanilla

按需读取

避免全量物化

→

v1

去重与复用

减少重复生产

→

v2

并行供给 + 自适应

降低访问与管理成本

汇报顺序

问题 → v1 → 新瓶颈 → v2 架构与算法 → 实验 → 耗时图 → RSS 图

优化对象不是数据集名称，而是重复率、访问成本、工作集大小和资源压力。

按需读取省下内存，却把特征生产放进训练循环

Vanilla 已有基础 SQL / raw cache；问题是重复生产和串行供给仍然昂贵。

采样节点
同一节点可能重复出现

→

SQL + 解码
重复访问与数据转换

→

文本 + Frame
重复编码与组装

→

GPU 更新
等待 ready 数据

问题 A ｜重复工作

同一节点、同一文本被多次处理；

原始行缓存不等于最终特征复用。

问题 B ｜供给不足

SQL 、编码和组装阻塞训练；

端到端耗时还包含首次初始化。

先减少每个 batch 的重复工作，再提高特征生产的并行度。

把重复请求合并，只对缺失部分生产特征

三层复用：节点请求、文本向量、已编码特征。

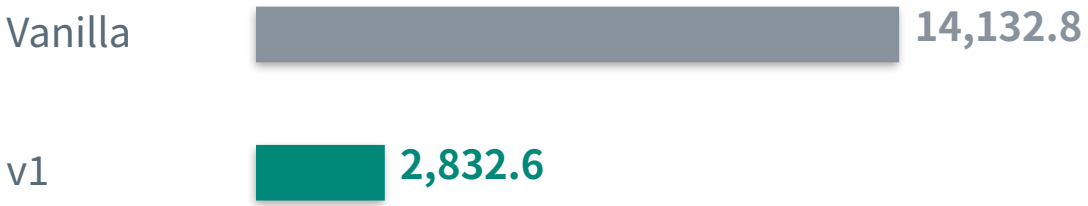
改动	省掉什么	必须保留什么
节点去重 + 跨 batch 窗口	重复 SQL / 解码 / 编码	恢复每个 batch 的节点位置和顺序
稀疏 ID 查询 / 稠密块读取	无效扫描和过量取数	相同请求行与时间约束
raw → text → encoded 缓存	逐步省掉后续转换工作	仅缓存静态特征、冻结文本向量
direct 文本路径 + 有界线程流水线	通用调度开销，重叠 SQL 与编码	兼容回退；限制队列与工作集

v1 的主要收益来自“少读、少转、少编码”，不改变训练输入的数量。

重复结构不同，同一缓存策略的收益就不同

历史单轮结果： Salt 的节点复用高， Stack 的节点复用低。

Salt： Vanilla → v1



4.99× • 秒

节点重复因子 2.114

raw 行覆盖率 98.70%

Stack： Vanilla → v1



1.22× • 秒

节点重复因子 1.011

raw 行覆盖率 4.86%

重复率决定“能省多少”；缓存查询和复制成本决定“省下来是否划算”。

命中缓存以后，访问成本仍可能成为瓶颈

增加缓存和 worker ，并不必然让训练更快。

高命中仍然慢

HM / Salt

整块 RPC 、共享锁、高频查表

工作集超出内存

Arxiv / Stack / Trial

缓存填满后反复查询、跨轮重算

并发资源持续增长

Stack / Amazon

mmap 数量、 WAL 、刷盘等待

准备工作压倒训练

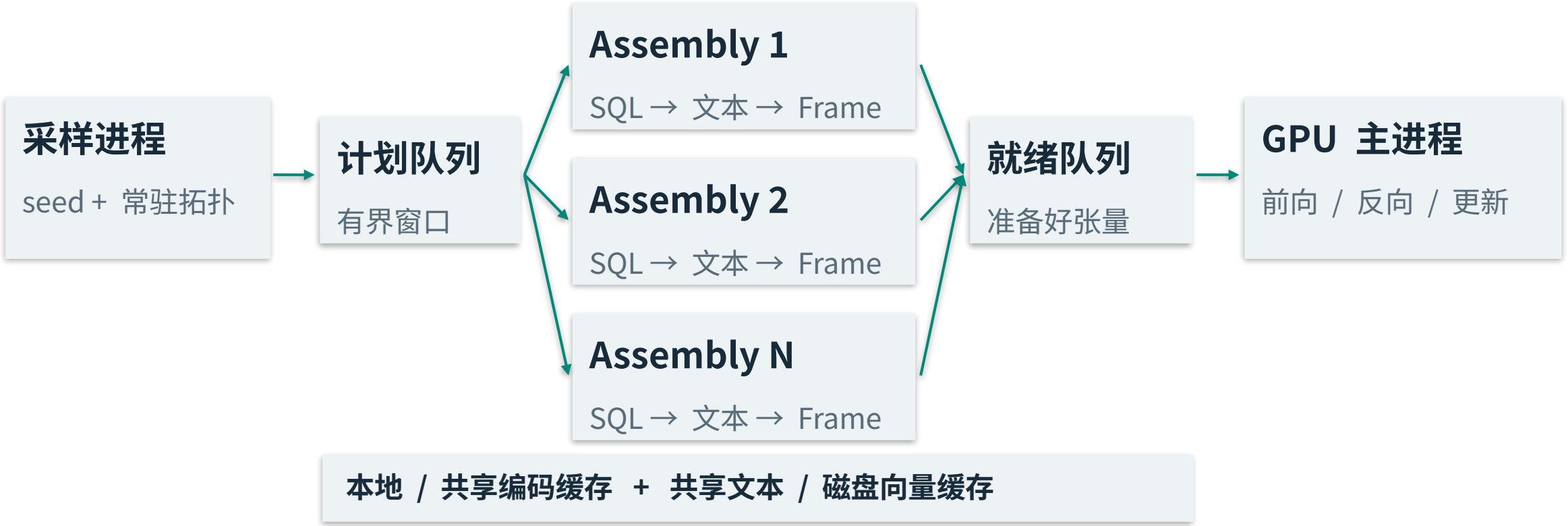
Event / F1

宽表冷统计；小任务启动成本

v2 需要同时解决：供给并行度、命中成本、缓存容量和长期运行资源。

采样、特征组装、 GPU 训练分进程执行

v1 单进程优化后， assemble 供给仍受限； v2 将三个阶段拆分并重叠执行。



队列与在途窗口受限； ready 顺序减轻队头等待，但会改变 batch 更新顺序。

按可观测信号决策，而不是按数据集名称分支

以下区分“自动策略”与“实验选定配置”。

信号	策略	代表数据集
高文本命中 + 查询代价高	启用本地槽位索引，绕过共享查询	Salt
大库 + 低内存命中 + 编码昂贵	启用持久化 embedding；复用已有缓存	Arxiv / Stack / Trial / Amazon
索引低复用 / 收益变小	退出本地索引，冷却后再评估	Arxiv 等低复用阶段
数值列可装入受限缓冲	按行数与预算分组，精确内存统计	Event；小表也适用
写入 / 读取阶段变化	调整 WAL 回收预算	Amazon 暴露的问题
编码缓存 RPC 昂贵	只返回请求行；可选择本地作用域	HM：本地 4 GiB 是选定配置

F1 / Avito 不必强开磁盘缓存； Ratebeer 也不能仅因“大”就自动解决编码瓶颈。

重复命中时，只查本地槽位索引

向量仍共享；本地仅保存“文本 → 已发布槽位”，避免复制整份向量。

观察窗口

64 次调用

≥1,024 次请求

→

自动激活

共享命中 ≥ 80%

查询累计 ≥ 5 ms

→

本地命中

直接批量 gather

跳过共享哈希与锁

自动退出

连续两个窗口本地命中 <20%：

释放索引，并进入冷却期。

18,260 → 5,208 s

Salt 两轮完整耗时；模型输入不变

共享查询由约 30,532 降至 802 worker 秒

高命中率任务的重点是降低命中成本，而非继续增加编码能力。

把跨轮可复用的 embedding 留在磁盘

本地索引 → 共享文本缓存 → 磁盘缓存 → 真正缺失才编码。

本地槽位
命中即返回

→

共享向量
命中即返回

→

磁盘向量
命中即返回

→

GloVe 编码
发布结果并复用

新缓存的启用条件

数据库 ⩾ 256 MiB； 模型输入 ⩾ 4,096；

模型耗时 ⩾ 1 s； 内存命中率 ⩽ 10% 。

已有兼容磁盘缓存可直接复用。

观察到的收益

Arxiv： 第二轮编码显著减少。

Trial： 热缓存复测两轮编码均为 0 。

F1： 小数据集不激活磁盘路径。

冷构建要付编码与写入成本；热缓存成绩必须与冷启动成绩分开。

只传需要的行，避免命中后搬运整块特征

先优化通用共享路径，再用对照实验选择本地或共享作用域。

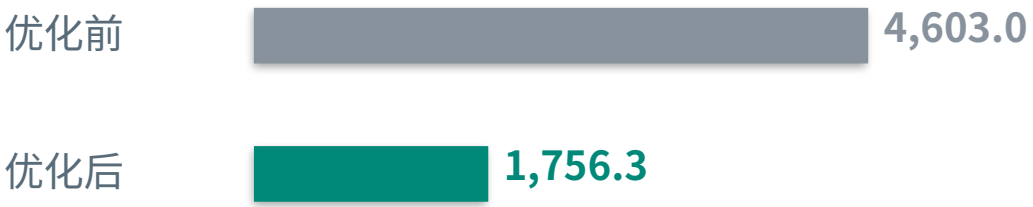
原共享路径

少量请求 → 整块 TensorFrame RPC → 再切片

优化后

返回请求行； HM 使用 worker 本地缓存

HM： 两轮流水线



2.62× • 秒

等待 ready： 3,348.9 → 481.9 秒

文本编码、 SQL 计时基本不变。

使用本地总计 4 GiB 、 4 worker ；

这是验证过的配置，不是自动决策。

缓存能命中，还要能长期稳定运行

缩短临界区、限制映射数量，并随读写阶段调整日志回收。

Stack ｜查询与映射

- 哈希、 gather 移到锁外，批量发布。
- mmap 最多保留 1,024 个， LRU 回收。
- 共享查询短测： 296.0 → 53.5 秒。
- 完整两轮： 3,665.66 秒。

Amazon ｜ WAL 与写入

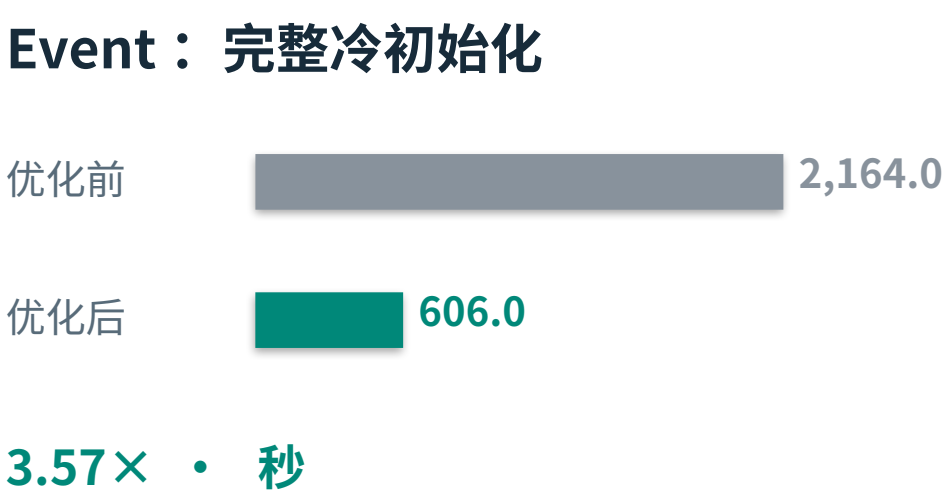
- 每 writer 追加 8,192 行固定分片。
- 预算参考索引大小和剩余磁盘。
- 读多写少时降低 WAL 回收阈值。
- 避免固定小阈值造成频繁刷盘。

WAL 阈值是回收触发条件，不是硬上限； Amazon 完整成绩未包含后加读阶段策略。

根据行数与内存预算，自动合并数值列扫描

不按 Event 特判，不抽样；精确统计结果保留。

1 估算单列缓冲：行数 × 8 字节
2 分组列数 = min(32, 128 MiB / 单列大小)
3 合并读取，批次最多 8,192 行
4 内存 partition 选取精确分位点
5 单列超预算 → 回退流式 SQL 路径



Schema： 1,895.62 → 335.66 秒；图拓扑： 267.95 → 269.92 秒。

329 项统计核对：分位数完全一致；均值 / 标准差最大绝对差约 8×10^{-13} 。

总初始化减少 72.0%（3.57×）；剩余约 44.5% 时间在图拓扑构建。

Ratebeer： 先修复发布竞争，再评价吞吐

并发修复解决数据损坏，不应直接包装成性能加速算法。

旧发布顺序

索引可见 → RPC 搬迁 storage

并发读可能访问旧地址

新发布顺序

先完成共享 storage → 再发布索引

后续序列化不搬迁已发布对象

验证：独立协调复现 + 回归测试 + 两轮各 10,000 batch 完成。

保留失败输入、精确 batch key 和类别校验；不截断坏值、不跳过样本。

仍有大量文本编码、 SQL 和组装开销，磁盘策略未启用。

首个 CUDA 异常未保存输入，不能确认与后续已复现竞争完全同源。

十项已有有效成绩，但并非统一最终版本重测

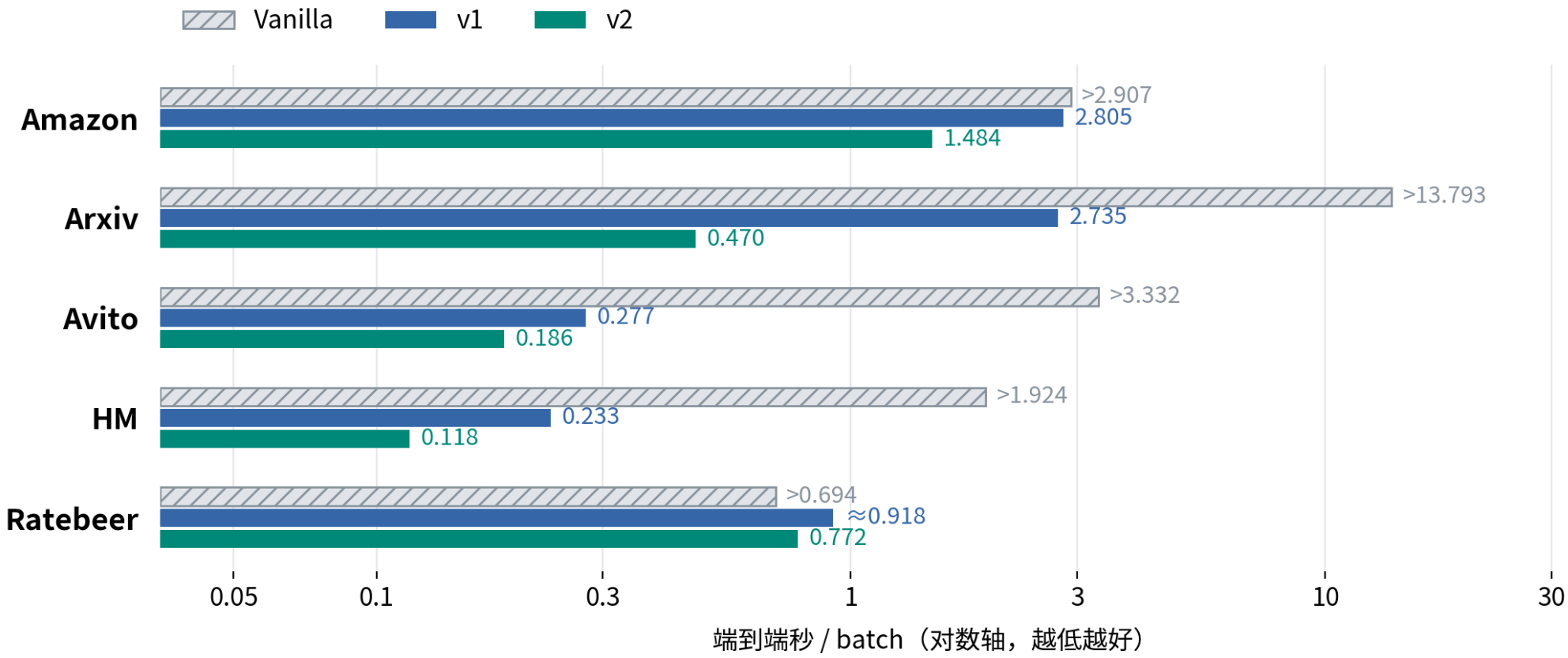
最新 v2 结果： Event / F1 用初始化优化后验证， Trial 用热缓存复测，其余沿用矩阵。

数据集	完成批次	端到端 / s	s / batch	解释条件
Amazon	9,908	14,699.63	1.484	部分热文本；诊断缺失
Arxiv	2,088	981.64	0.470	已有磁盘 embedding
Avito	8,644	1,604.83	0.186	磁盘文本关闭
Event	76	38.77	0.510	热初始化；非冷启动
F1	12	28.11	2.342	空初始化缓存验证
HM	14,972	1,760.13	0.118	本地编码缓存 4 GiB
Ratebeer	20,000	15,444.93	0.772	每轮截断 10,000 批
Salt	6,340	5,208.23	0.821	adaptive 索引复测
Stack	13,228	3,665.66	0.277	已有磁盘 embedding
Trial	918	371.80	0.405	初始化 + 文本热缓存

初始化优化： 59 项测试通过； Event / F1 两轮训练通过。跨版本训练等价仍需独立验收。

大规模任务：对比 Vanilla 、 v1 与 v2

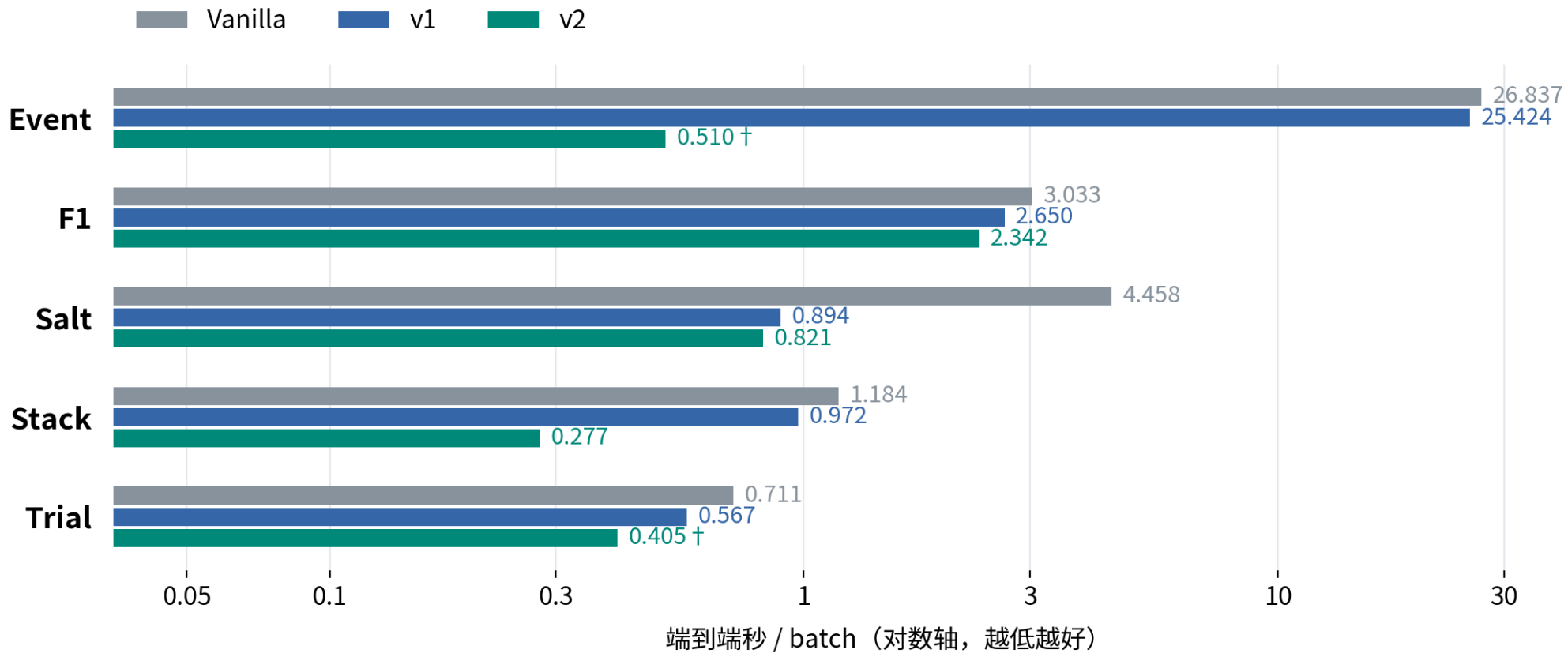
统一为端到端秒 / batch ； 历史 1 epoch ， v2 2 epoch ，启动成本摊销不同。



斜线 Vanilla 条为超时下界（ > ）； Ratebeer v1 为超时吞吐估算（≈）。

缓存与初始化影响，必须和吞吐一起解释

秒 / batch 越低越好；跨实验比较展示观测结果，不单独证明算法因果收益。



† Event 是热初始化训练， Trial 复用文本缓存； Event 冷初始化独立提升为 3.57× 。

用局部对照解释收益，而非只看版本排名

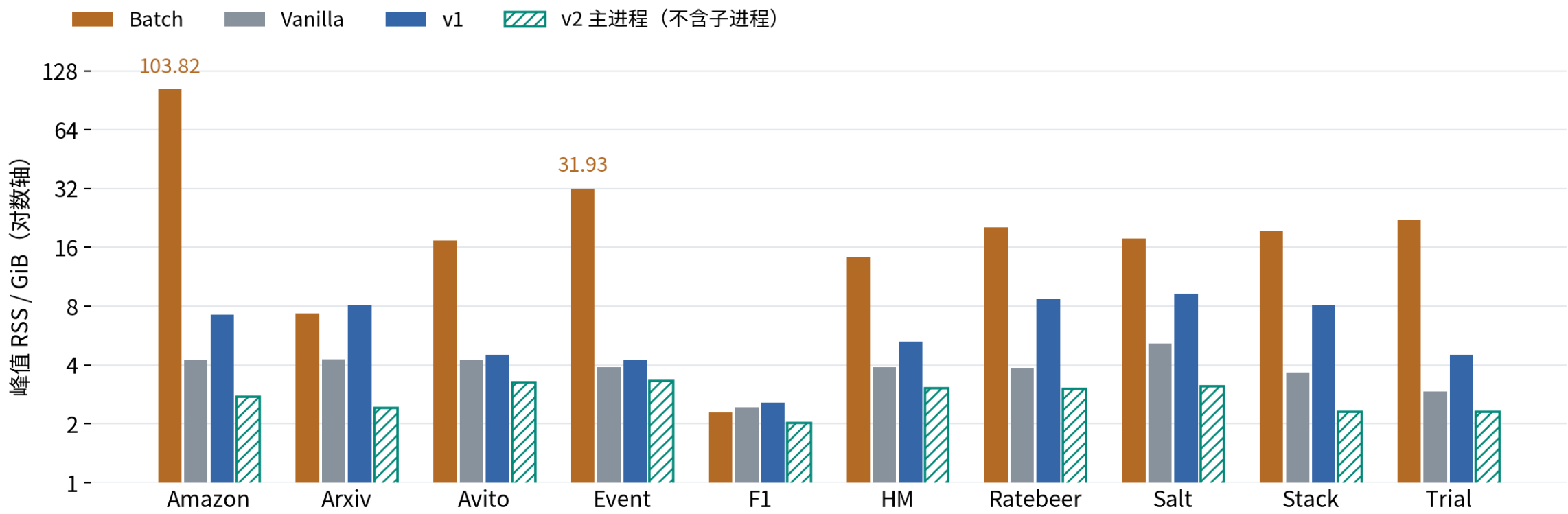
不同实验各有边界，不能把局部倍数直接相乘。

瓶颈	对照证据	能说明什么
HM： 跨进程传输	流水线 4,603 → 1,756 秒	减少整块 RPC / 本地作用域有效
Salt： 共享文本查询	完整两轮 18,260 → 5,208 秒	模型输入不变，命中路径成本下降
Stack： 共享查询临界区	同规模短测 296.0 → 53.5 秒	批量化与锁外处理改善缓存访问
Event： 精确数值统计	冷初始化 2,164 → 606 秒	空缓存下确有算法收益，结果一致
Trial： 已有文本缓存复用	两轮 656.95 → 371.80 秒	收益包含热缓存，不能全归因于改代码

观察：选定 v2 成绩均低于历史 v1；因果结论优先依赖明确的阶段对照。

全量物化的内存代价明显； v2 总内存尚待补测

实色：历史报告峰值 RSS；斜线： v2 主进程 RSS，不包含采样 / 组装 / manager。



不能由 v2 主进程约 2–3.3 GiB 推断整机占用，也不据此计算相对 Batch 的节省比例。

减少常驻特征，还需要覆盖整个进程树

缓存预算、队列上限与主进程 RSS ，都不等于系统总内存。

训练主进程

模型 / 优化器 / 当前 batch

采样 + manager

拓扑 / 索引 / 共享缓存

Assembly × N

GloVe / SQL / 临时 Frame

测量补齐：进程树 PSS / USS 、 cgroup 峰值、显存与磁盘资源。

性能优先级： Amazon / Ratebeer 的特征供给； Event 的图拓扑构建。

最终总评：固定代码和资源预算，分别做冷缓存与热缓存十库复测。

主线不变：少做重复工作 → 提高供给 → 让命中更便宜 → 控制资源与冷启动。

v2：相对 Vanilla 提速，相对 Batch 权衡

逐数据集计算相对变化，再等权取算术平均；耗时统一为端到端秒 / batch 。

相比 Vanilla

已记录 RSS 平均降低 27.5%；在双方均完成的 5 个数据集上，单位 batch 耗时平均降低 64.4% 。

相比 Batch

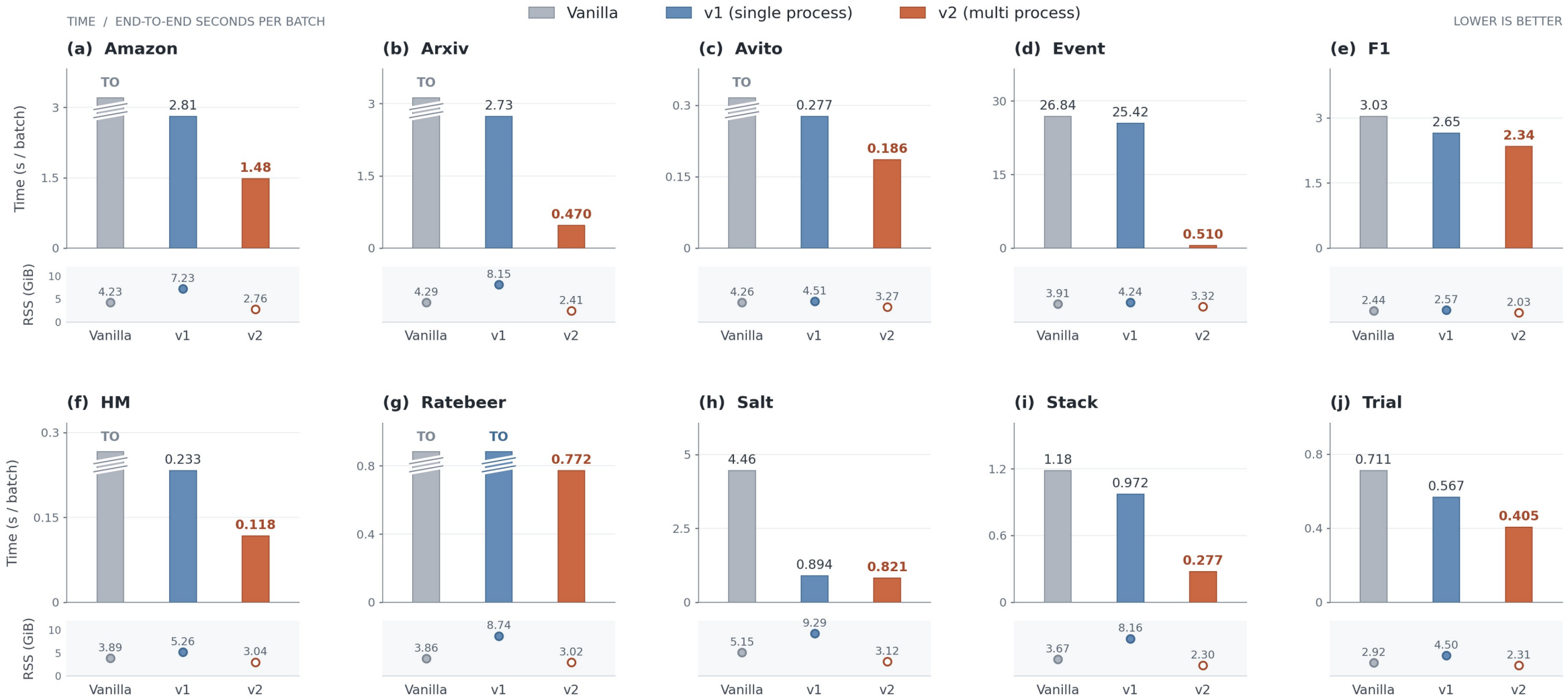
Batch / v2 的 RSS 记录值比平均为 9.18 倍；v2 单位 batch 耗时平均增加 75.3%（为 Batch 的 1.75 倍）。

RSS 口径限制： v2 仅统计主进程，不含采样、组装和 manager；上述比值不代表整体内存降幅。

RSS 与 Batch 耗时： 10 库； Vanilla 耗时： Event 、 F1 、 Salt 、 Stack 、 Trial ，剔除其余 5 项超时。
历史超时 RSS 为终止前峰值； v2 为两轮、缓存状态混合， Ratebeer 截断；结果不是严格配对实验。

来源：历史实验报告 / rss-data.csv / performance-data.json ；未使用本次内存估算。

十数据集性能对比｜Vanilla · v1 · v2



逐数据集解释（1/2）｜瓶颈决定优化方式

左侧倍率： v1 / v2 的单位 batch 耗时；右侧列出证据与解释边界。

Amazon **1.89×**	**大量文本与特征供给限制训练吞吐** 并行组装＋持久化 embedding，减少重复文本模型调用。	部分热缓存； worker 诊断缺失， 不能量化各优化的独立贡献。
Arxiv **5.82×**	**长文本、低复用，组装速度拖慢 GPU** direct GloVe ＋并行组装，磁盘向量跨轮复用。	模型输入： 184,724 → 1,316 ； 运行前已有部分磁盘缓存。
Avito **1.49×**	**SQL 与编码缓存访问是主要供给成本** 多进程重叠采样 / 组装 / 训练，复用文本与编码特征。	第二轮文本模型输入为 0 ； 仍有 544.8 秒 ready 等待。
Event **49.83×***	**宽表初始化很重，训练批数却很少** 预算分组扫描＋精确分位数；缓存避免重复初始化。	* 图中是热初始化结果；冷测 2,164 → 606 秒才是独立证据。
F1 **1.13×**	**数据小，启动与进程协调占比高** 通用缓存复用；门槛避免不必要的磁盘持久化。	改善有限； 28.11 秒为最新验证， 没有证据表明新增初始化优化提速。

RSS 图显示的是记录值； v2 仅主进程，不能据此断言整体内存接近或降低。

逐数据集解释（2/2）｜减少重复工作与命中开销

左侧倍率： v1 / v2 的单位 batch 耗时；右侧列出证据与解释边界。

HM **1.98×**	**共享编码缓存命中后仍传输整块数据** 改为返回请求行；本次使用 worker 本地编码缓存。	4 worker 总预算 4 GiB； ready 等待 3,348.9 → 481.9 秒。
Ratebeer **≈1.19×**	**工作量大，仍持续消耗 SQL 与文本编码** 多进程重叠供给；共享缓存发布修复保障长跑稳定。	不是 4 小时内完成完整两轮； 详见下一页的规模与预算对照。
Salt **1.09×**	**高文本复用，瓶颈转为共享查询与锁** worker 本地文本→槽位索引，批量提取共享向量。	相对早期 v2： 18,260 → 5,208 秒； 相对已经优化的 v1，收益较小。
Stack **3.51×**	**共享查询竞争，以及大量磁盘分片映射** 批量查询 / 发布、锁外处理， mmap LRU 控制映射数。	复用既有磁盘 embedding； 不能把全部差异归因于并行。
Trial **1.40×**	**命中后仍有共享查询与磁盘访问成本** adaptive 本地索引＋已存在的磁盘 embedding 复用。	两轮文本模型输入均为 0； 此次改善包含热缓存收益。

RSS 图显示的是记录值； v2 仅主进程，不能据此断言整体内存接近或降低。

Ratebeer ｜为何两个 TO ，只有 v2 有完成时间?

图中“完成”同时受吞吐、运行规模和预算影响，需要拆开解释。

Vanilla

4 小时超时

目标： 1 × 20,743 批
缺少可靠的完成批数记录

v1 (single process)

4 小时超时

完成： 15,686 / 20,743 批
观测吞吐约 0.918 秒 / batch

v2 (multi process)

4 小时 17 分完成

完成： 2 × 10,000 批（截断）
端到端 0.772 秒 / batch

性能：观测吞吐约快 1.19× ，单位 batch 耗时约少 15.9% 。

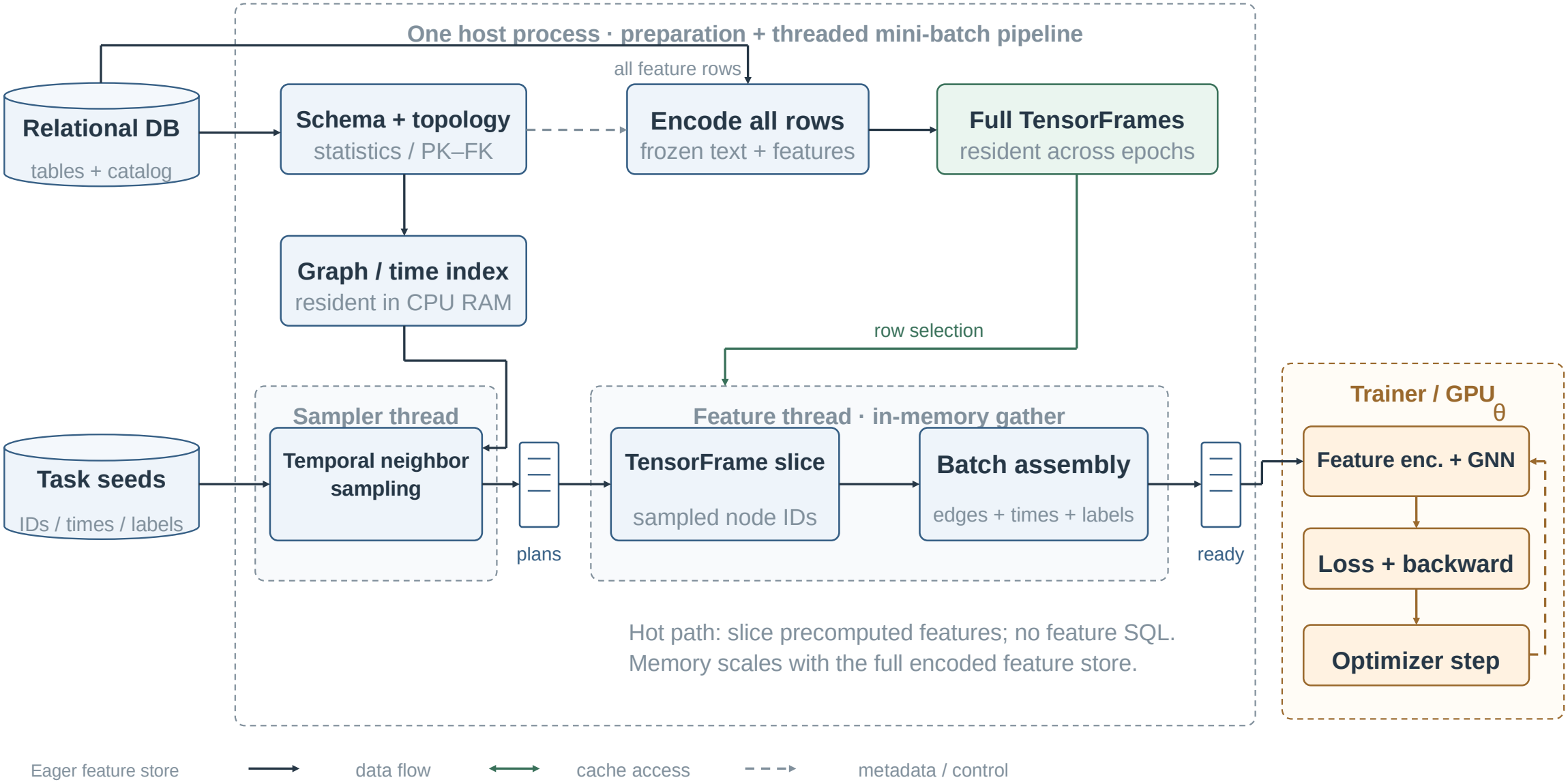
机制：并行采样、 SQL / 特征组装与 GPU 训练，改善供给；但 ready 等待仍达 11,389 秒，
两轮仍各有约 2,000 万条文本模型输入，说明 SQL 与编码瓶颈尚未消除。

稳定性：先将 TensorFrame 存储转为共享内存，再发布缓存条目，修复并发读取损坏；
修复后通过本次 20,000 批长跑。这是正确性证据，不能独立解释 1.19× 的性能收益。

结论： v2 在本次配置下更快且成功跑完；尚未证明在同一 4 小时预算完成完整两轮。

(a) Batch ｜全量物化训练架构

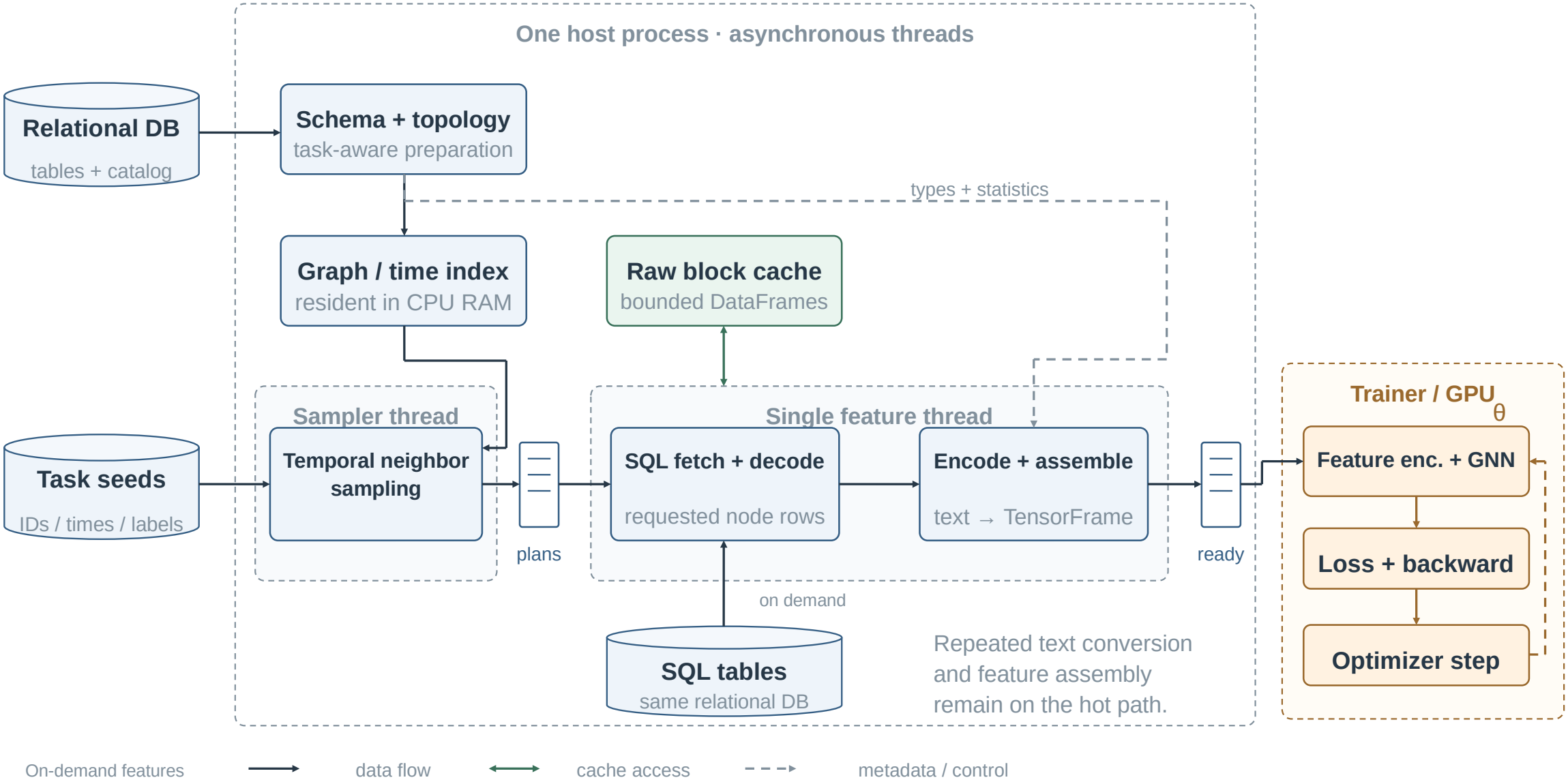
训练前生成全部特征；训练时从常驻 TensorFrame 切片。虚线框表示执行范围。



Batch: eager materialization followed by sampled mini-batch training. Static features are resident; model parameters remain trainable.

(b) Vanilla ｜按需读取的在线训练架构

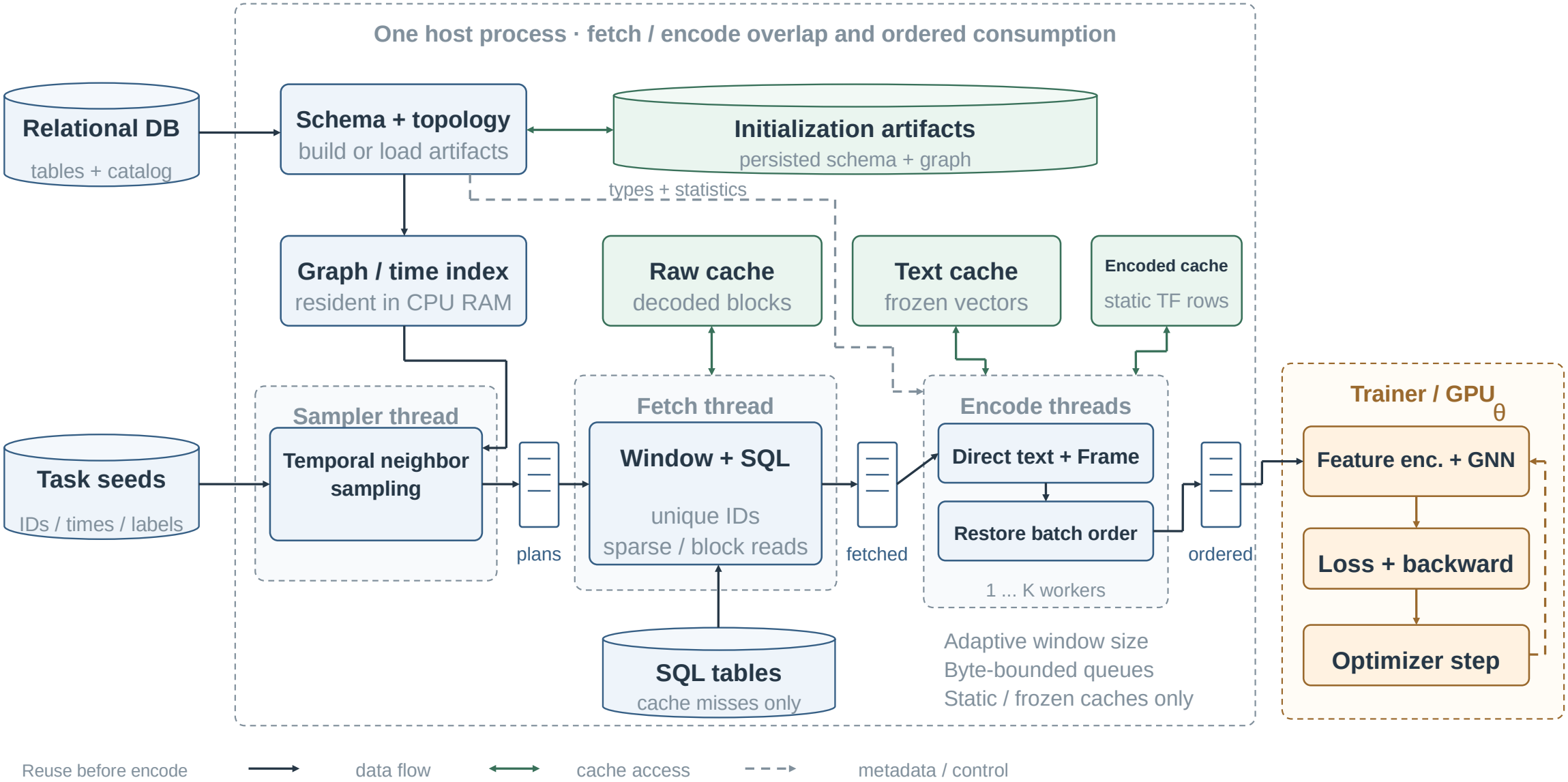
单进程内异步采样、特征线程与训练；已有 raw cache ，但缺少后续文本与编码特征复用。



Vanilla: bounded online feature access with a raw cache and asynchronous threads; static text / encoded-feature reuse is not yet added.

(c) v1 | 窗口复用与分层缓存训练架构

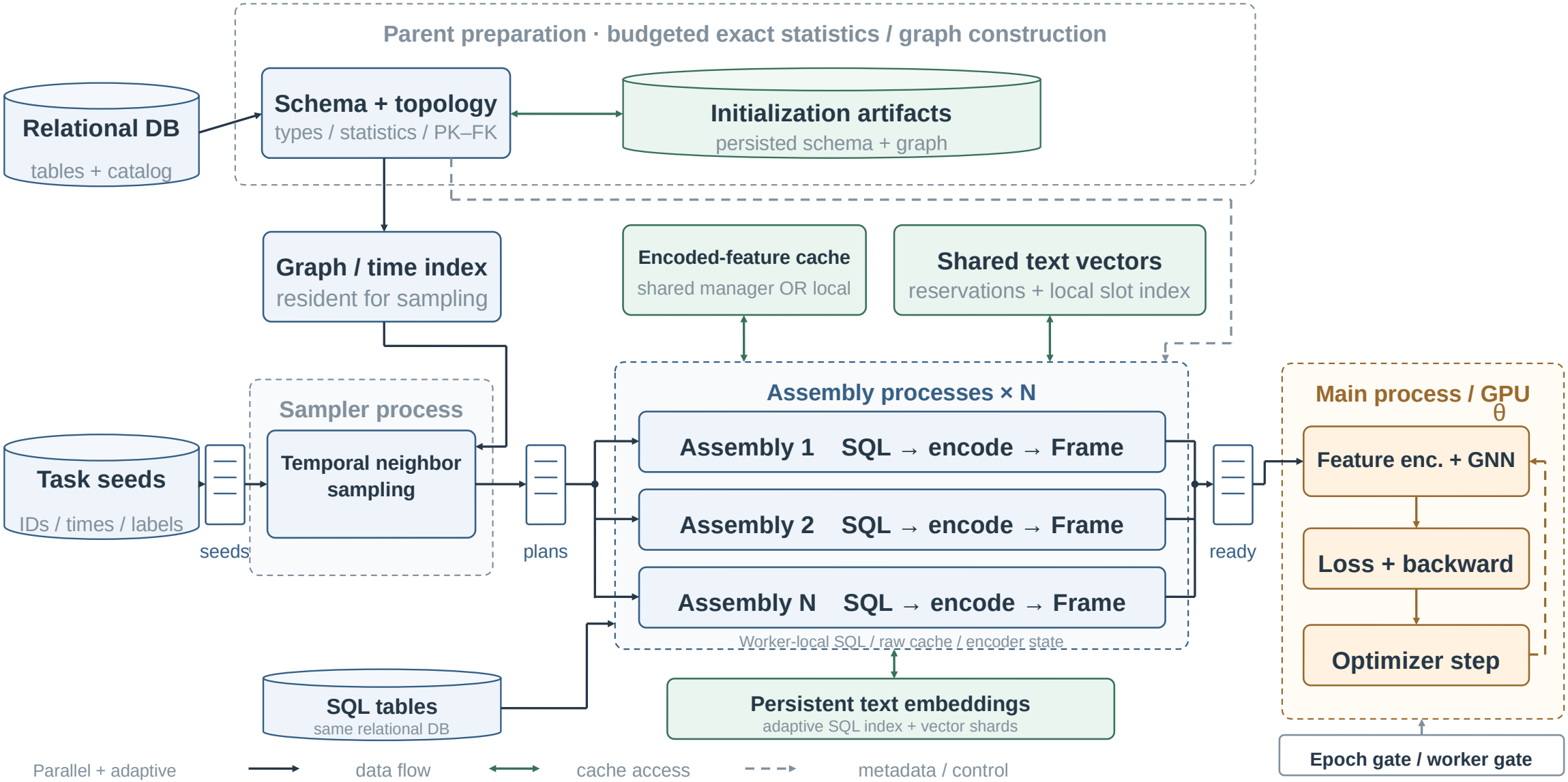
在单进程线程流水线中合并请求，并复用 raw 、 text 与 encoded 特征；训练按序消费。



v1: deduplicated feature windows, layered in-process caches, overlapping fetch / encoding threads, and restored per-batch inputs.

(d) v2 | 多进程与自适应缓存训练架构

跨进程传 SamplePlan 和张量； SQL 、字符串和 DataFrame 留在各组装进程。



v2: separate sampling and assembly processes, bounded IPC queues, shared/local feature caches, and adaptive persistent text reuse.